



中山大學
SUN YAT-SEN UNIVERSITY

OS 挑战赛 Proj59 (决赛)

SLIM-ARC: 面向端侧 Agent 的 低内存 MoE 推理运行时与可复现实验

队 名: 依托 Agent 答辩 OS

组 员: 欧阳易芃 马福泉 刘昊

指导教师: 赵帅 张献伟

学 校: 中山大学

赛 道: 操作系统功能挑战赛

选 题: Proj 59

日 期: 2026 年 8 月 17 日

目录

1	引言	4
2	背景、动机与边界	4
2.1	MoE 稀疏性不是自动的内存安全	4
2.2	相关工作的设计启发	4
2.3	决赛证据口径	5
3	决赛运行时设计	5
3.1	阶段感知的映射建议	5
3.2	层感知预取调度	5
3.3	统一预算：计算方式与生效范围	6
3.4	错误专家页回收：安全优先的候选建议	6
3.5	压力/准确性导向的专家驻留	7
3.6	不变量与设计取舍	7
4	生命周期、并发与可观测性	7
4.1	模型所有权与租约	7
4.2	并发边界	7
4.3	严格指标契约	7
4.4	实现边界与故障回退	7
5	受限 Mac 实验协议与结果导入	8
5.1	固定实验矩阵	8
5.2	记录与推广门槛	8
5.3	结果状态	8
5.4	赛前末次 A24 运行复核	9
5.5	跨设备边界实验	10
5.6	Ascend 910B4 最小兼容性验证	10
5.7	树莓派慢存储专项优化	11
6	端侧 Agent Harness：面向受限推理的工作负载闭环	12
6.1	实现路径	12
6.2	短上下文与慢输出自适应	13
6.3	受限 Shell 与可观测性	13
6.4	功能闭环验证	13
7	结论、局限性与未来工作	14
8	附录：可追溯性清单	16
8.1	初赛到决赛的里程碑	16
8.2	决赛阶段的主要问题与处理	16
8.3	提交材料与代码入口	17
8.4	实验可追溯性	17

8.5 报告构建保护	17
----------------------	----

摘要

端侧混合专家 (Mixture-of-Experts, MoE) 模型可将大量参数与每个 token 的实际计算解耦, 但受限内存环境仍会因模型映射、页缓存、运行时缓冲和低速存储之间的竞争而失效。本决赛阶段报告聚焦一个可检验的专家权重页管理闭环: 利用 Router 观测提出有界预取和驻留建议, 根据实际选择撤回错误建议, 并以不改变模型权重、路由结果、KV 内容或张量地址为安全边界。

系统以阶段感知的内存建议、层级预取、错误专家页回收和压力/准确性导向的专家驻留组成闭环。回收只针对未被本轮选中的预取专家、且完全落在专家张量内部的页; 驻留使用压力状态、预测浪费和饱和热度决定准入。运行时归属于模型而非进程全局对象, 图执行路径通过租约访问它; 销毁时先拒绝新租约并等待在途租约, 再停止工作线程和释放地址注册表, 从而使地址生命周期和并发边界可审计。

本文不把已失效的初赛跨设备、跨内存档位结果作为决赛性能结论。我们在固定 Mac/Colima、2 GiB、4 vCPU、no-swap、pp64/tg16 条件下完成两轮五配置 cold/warm 矩阵, 20 次运行全部成功。所有配置的聚合峰值均为 2.000–2.000 GiB, 因而本轮没有证明峰值内存下降; 组合策略的 cold wall time 为 79.525s, 相对 control 的 66.300s 回退 19.947%, 最终被拒绝。RK3588 和树莓派实验进一步说明: 静态页建议与提前文件预读不能跨设备直接复用, 策略必须匹配存储链路和访问阶段。在树莓派慢存储 campaign 中, 常驻 shared/hot 专家路径将 decode throughput 相对 A24 提高 14.16%, 16 MiB 统一预算又在吞吐近似不变时将投机 weight I/O 降低 98.44%; 这些结果只在该设备内推广。所有正式数值都绑定原始日志或结构化证据, 避免将机制正确性或随机波动误写为优化收益。

关键词: 端侧 LLM 推理; MoE; 虚拟内存; 专家页回收; 自适应驻留; 可复现实验

项目链接 (P2) <https://www.bilibili.com/video/BV1fXTF6HEAw?p=2>

1 引言

MoE 以稀疏路由减少单 token 激活的专家数，但低内存机器仍必须在映射权重、页缓存、KV cache 和临时缓冲之间做出及时而安全的取舍。已有端侧工作说明，权重卸载、内存保留和异步数据移动是重要的设计空间：FlexInfer 探索了张量级卸载与保留 [1]，PowerInfer-2 面向智能手机讨论了端侧大模型执行 [2]，MoE-Prism 则表明模型与系统协同能够改变专家服务的资源弹性 [3]。这些工作构成问题和设计动机；它们并不证明 SLIM-ARC 的实现或性能。

决赛阶段的问题更具体：当 Router 的短期预测与实际选择不一致，如何撤回不再有用的页建议；当内存压力升高时，如何只为有足够证据的专家保留有限预算；以及如何证明建议目标在并发和模型析构下仍然有效。把这些问题仅当作启发式调参，会把错误的地址、重复工作或悬空后台任务带入推理路径。

SLIM-ARC 因而将核心贡献表述为一个受约束的专家权重页管理闭环：阶段识别 → Router 预测 → 预算准入 → 预取 → 实际观测 → 回收或驻留。这比单独将 MADV_RANDOM 或“专家预取”称为核心创新更准确，因为性能取决于访问阶段、内存压力和存储设备的共同作用。本文贡献是：

- 给出阶段感知的映射建议和层级预取路径，并在 RK3588 上用静态策略的负结果说明页建议必须随设备与阶段调整；
- 给出一套仅对完整内部页操作的错误专家回收计划，并将非法布局、非法 ID、跳过字节和建议失败显式计数；
- 给出压力、错误率和热度共同约束的专家驻留策略，所有新行为保持 opt-in，未通过性能门槛时不成为默认路径；
- 用模型所有权、租约和有界工作队列约束生命周期与并发，并通过严格运行时指标将机制行为连接到每次实验；
- 在 Mac 上完成 20 次身份绑定运行，并在 RK3588 与树莓派上报告阶段建议和慢存储策略的适用边界；树莓派进一步形成“关闭投机预取、常驻 shared/hot 专家、预算约束、跨 token LRU”的设备策略，不把不同设备合并为单一加速比。

2 背景、动机与边界

2.1 MoE 稀疏性不是自动的内存安全

MoE Router 每步只选择少量专家 [4, 5]，但模型文件的映射粒度、操作系统页缓存和预取粒度未必与该选择一致。因而，“只激活少量专家”只能说明存在降低无效 I/O 的机会，不能推出任何固定内存占用或加速比例。SLIM-ARC 通过标准的页建议接口表达候选动作；该接口本身不改变权重、路由输出或采样语义。

2.2 相关工作的设计启发

FlexInfer 以异步预取、内存锁定和灵活张量保留拓展端侧推理的内存设计空间 [1]。PowerInfer-2 说明端侧设备上的高效模型执行需要系统级资源管理 [2]。MoE-Prism 进一步将专家的系统弹性与模型结构协同讨论 [3]。EcoSpec 从 MoE 投机解码角度指出，新增专家会产生离散的边际加载成本，并据此选择草稿长度 [6]；KTransformers 则展示了异构执行、热冷专家放置和异步流水线的价值 [7]。

SLIM-ARC 不实现 EcoSpec 的 draft model, 也不复现 KTransformers 的 AMX、NUMA 或 GPU/NPU 算子路径。我们只吸收其中可移植的系统原则：专家预测必须同时考虑收益和新增 I/O 成本；在慢盘或极低内存下，如果额外专家预取的边际收益不足，就精确关闭专家 WILLNEED，保留 demand paging、Router 观测和其他运行时路径。该原则对应树莓派 A24-A33 的投机预取开关与统一预算，而不是新增一个未经验证的性能模块。本项目也不复现或比较这些论文的数值；相关工作只支持设计动机，性能结论仍来自第5节的本项目证据。

2.3 决赛证据口径

初赛报告中来自不同设备、内存档位、缓存状态、工作负载或未完成镜像链接验证的行，均不能与本阶段的受限 Mac 运行相除或合并。已经被审计否定的历史性能叙述不出现在本报告中。决赛报告将把机制正确性、设计动机和最终性能证据分开：只有最后一类能产生吞吐、内存峰值或提升比例的结论。

3 决赛运行时设计

3.1 阶段感知的映射建议

一次性对整个模型设置固定页建议并不可靠。Prefill 通常按层扫描较大范围，连续预读可以摊薄缺页与存储请求开销；Decode 只激活少量专家，但在内存远小于模型且存储随机访问较弱时，彻底关闭预读反而会把大块顺序 I/O 分裂为同步小读。SLIM-ARC 因而把页建议作为设备策略：加载与 Prefill 使用 SEQUENTIAL；Decode 在随机按需和保留预读之间由设备配置选择。该机制只改变内核 readahead 提示，不改变计算图或 Router 结果。

3.2 层感知预取调度

层级预取只覆盖本轮计算图及其有限前瞻窗口。设 $l_{\min}$ 和 $l_{\max}$ 为计算图中出现的最小、最大 Transformer 层， w 为随 Prefill/Decode 阶段变化的前瞻层数， T_l 为第 l 层已注册的 mmap 权重张量集合。算法1给出调度路径；循环上界已经截断到 $l_{\max}$ ，因此不再保留旧稿中永远不可满足的 $l > l_{\max}$ 分支。

Algorithm 1 有界层感知预取

输入：计算图 G ，阶段相关窗口 w

输出：对有限后续层发出 best-effort 页建议

```

1:  $(l_{\min}, l_{\max}) \leftarrow \text{layer\_range}(G)$ 
2:  $l_{\text{end}} \leftarrow \min(l_{\max}, l_{\min} + w)$ 
3: for  $l = l_{\min}$  to  $l_{\text{end}}$  do
4:   for all  $t \in T_l$  do
5:      $\text{madvise}(t.\text{addr}, t.\text{size}, \text{WILLNEED})$ 
6:   end for
7: end for
```

Router 观测不通过裸指针跨线程共享。每次预取分配单调递增的 generation token，并把层号、专家集合和成功建议字节作为不可变记录发布；实际 Router 结果只结算同 token 的记录。计算失败、缺少 Router 节点或过滤后集合为空时显式取消 token，从而避免旧预测占满有界队列。

3.3 统一预算：计算方式与生效范围

调度器先从静态周期额度 B_{static} 出发；压力准入开启时，根据 cgroup 当前占用、上限和保留量得到有效额度 $B_{\text{eff}} \leq B_{\text{static}}$ 。随后按阶段系数 $\alpha_p = (\alpha_w, \alpha_{kv}, \alpha_e)$ 计算

$$(B_w, B_{kv}, B_e) = B_{\text{eff}} \alpha_p, \quad \alpha_w + \alpha_{kv} + \alpha_e = 1. \quad (1)$$

运行时统计可在有限范围内调高发生 stall 的路径，并缩减另外两路。当前实现采用启发式乘法调整，没有在每次调整后对三路比例做二次归一化；因此这些额度应解释为各路径的 admission 上限，而不是三路实际 I/O 字节和严格等于 B_{eff} 。表1列出当前初始系数。

表 1: 统一预算的阶段初始比例（策略参数，不是测得带宽占比）

阶段	权重 α_w	KV α_{kv}	专家 α_e
PREFILL_SHORT	60%	10%	30%
PREFILL_LONG	50%	20%	30%
DECODE_SHORT	70%	20%	10%
DECODE_LONG	30%	60%	10%
MOE_DECODE	20%	20%	60%

必须区分“计算出额度”和“严格执行额度”。当前权重预取通过 `set_memory_budget` 执行 B_w ，专家路径在压力准入、驻留或专家预算开关启用时执行 B_e ；决赛的 model-owned runtime 尚未绑定 KV manager，因此 B_{kv} 只存在于策略计算中，没有进入正式实验路径。即使后续绑定 KV manager，现有接口也尚未把 B_{kv} 作为逐周期硬 admission。因此本报告将其称为统一预算策略与两路执行路径，不宣称权重、KV、专家三路已经完成同等强度的带宽隔离。补齐 KV manager 绑定、字节级 admission 和比例归一化后，才能把表中 KV 比例解释为实际约束。

3.4 错误专家页回收：安全优先的候选建议

每轮 Router 观测产生已预取集合和实际选中集合。回收规划器保留前者中未出现在后者的 ID，并保持首次出现顺序。对于每个候选专家，它验证张量的专家数、可整除布局、ID 范围和无溢出的偏移计算；再把专家切片向内收缩到完整页面。一个专家通常跨越多个页面，边界页还可能与相邻切片共享，因此系统只对完全位于该专家切片内部的页面提出建议。零长度、跨越张量边界、非整除布局或非法 ID 都不产生页面建议，而是记录可审核的原因。这样，建议绝不覆盖选中专家，也不触及专家切片外的字节。

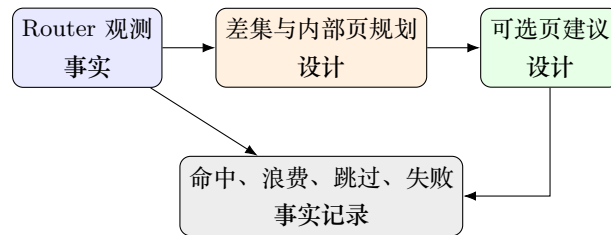


图 1: 安全错误专家页回收闭环（设计图，非性能测量）。事实记录由严格运行时指标输出；是否带来性能收益必须由第5节协议的正式结果判定。

3.5 压力/准确性导向的专家驻留

驻留不是无条件缓存。运行时从 `cgroup v2` 读取 `memory.current` 与 `memory.max`，将压力归为 `normal`、`high`、`critical` 或 `missing`；无效、溢出或不一致读数走显式兼容回退，而不被当作剩余容量。策略同时参考预测浪费的 EWMA 和饱和的专家热度：高压时收紧准入，错误率高时不为不可靠预测预留预算。压力阈值、恢复所需的连续样本和预算均为可测试的确定性规则，而非从最终结果反推的参数。

3.6 不变量与设计取舍

回收与驻留只发出建议，建议失败记为指标并回退，不使推理失败。其代价是保守：页边缘和不规则张量可能被跳过，错误预测也可能使驻留无收益。该取舍优先于以任何性能代价换取覆盖率；是否满足推广门槛留给受控实验决定。

4 生命周期、并发与可观测性

4.1 模型所有权与租约

运行时作为 `llama_model::impl` 的成员，在模型映射和张量注册完成后激活，而不是作为进程全局调度器。图执行 `hook` 获取 `move-only` 租约；它不保存裸调度器指针。析构顺序为：从全局获取注册表移除运行时、拒绝新租约、等待在途租约、停止并 `join` 预取线程、发出最终指标、销毁地址注册表，最后才允许模型映射销毁。这一顺序的目标是避免有效模型/上下文生命周期内的 `use-after-destruction`，而不是宣称对违反上游生命周期约定的调用提供保护。

4.2 并发边界

工作请求是有界的 (`generation`, `layer`) 队列。工作线程在队列锁内认领请求，然后在不持有调度器互斥锁时执行 `posix_madvise`；相同 `generation` 的请求不能被两个 `worker` 重复认领。队列满时丢弃最旧的未认领请求并计数。这将高负载时的行为从无界积压变成可观测的降级。

4.3 严格指标契约

每个成功 `patched benchmark` 进程只允许一条带显式版本号的 `[SLIM-ARC-RUNTIME]` 行，记录专家发出/命中/浪费字节、页建议请求与合并区间、回收候选/调用/跳过/失败、驻留准入/回退和压力计数。解析器按对应版本拒绝重复行、缺失字段、未知字段、非十进制和溢出；`baseline` 必须没有该行。这样可以同时保留旧实验所绑定的历史 `schema` 和当前扩展后的 `schema`，而不会把不同版本的字段静默拼接。机制测试可验证这一契约，但不能替代真实模型上的性能实验。

4.4 实现边界与故障回退

页建议均在状态锁外执行，返回值逐次计数；失败只取消本次建议，不改变计算结果。权重预算不足时跳过超额范围并累加 `skipped bytes`，专家 `generation` 无法发布或队列已满时不消耗预算。`cgroup` 读数缺失、上限为无限、当前值大于上限或发生算术溢出时，压力策略进入 `missing/fallback` 状态，不把异常读数解释为可用内存。上述回退保证 `opt-in` 优化失败时仍退回原始推理路径。

5 受限 Mac 实验协议与结果导入

5.1 固定实验矩阵

正式性能证据取自固定 Mac/Colima 环境: cgroup v2 `memory.max=2 GiB`、四个 benchmark vCPU、`memory.swap.max=0`、pp64、tg16、相同 prompt、seed 和上下文。冻结模型为 Qwen3-Next-80B-A3B-Instruct-Q4_K_M.gguf, 文件大小为 48,410,988,384 B (约 45.09 GiB), SHA-256 为 d103b2733ec1012a52d01edda66b7e5c24ae50508c9f99f5297ea459ef3c061a; 冻结 llama.cpp revision 为 360e134。实验按 baseline、patched-control、patched-reclaim、patched-residency、patched-combined 执行; 恰好两轮, 每轮中每个配置各有一次 cold 和一次 warm, 共 20 次运行。全部运行成功, 且没有用重试结果覆盖失败行。

5.2 记录与推广门槛

每个接受行必须绑定本地 commit、完整 patched source hash、模型 SHA-256、镜像 ID 与变体链接、资源限制、缓存标签、wall time、吞吐、cgroup peak、OOM/swap/fault/I/O 计数、运行时指标和原始运行目录。回收仅在降低稳定档位, 或将 `memory.peak` 降低至少 10% 且 wall regression 不超过 15%、major faults 与 read bytes 不超过 control 的两倍时, 才可能 promoted。驻留需在浪费、storage reads 或 decode throughput 中至少一项改善 10%, 并满足相同稳定档位与 wall regression 约束。combined 还必须优于 control 和至少一个单项最终候选。否则分类为 kept_opt_in、rejected 或 insufficient_evidence。

5.3 结果状态

两轮矩阵已完成。第2表显示, 所有配置的聚合峰值都落在 2.000–2.000 GiB; 在该 cgroup 上限已经饱和的实验中, 不能声称任何机制降低了峰值内存。cold 条件下 patched-control 相比 baseline 的 wall time 和 decode throughput 均改善, 而 warm 条件下方向相反, 说明缓存状态足以改变排序, 不能把 cold/warm 混合为一个加速比。

单项机制呈现的是“可保留但不能推广”: reclaim 与 residency 在总体 gate 中均为 kept_opt_in。这类分类只说明它们没有越过拒绝边界, 并不证明机制动作导致了表中波动。相反, combined 的 cold wall time 为 79.525 s, 相对 control 的 66.300 s 回退 19.947%; cold gate 和总体决策均拒绝组合策略。这一负结果表明, 两项保守策略叠加会增加 fault/admission 路径上的成本, 单独看机制正确性不能推出协同收益。

JSON 的生成、核验和填表步骤记录于 RESULTS_IMPORT.md。正式构建驱动每次读取固定路径 JSON, 重新生成并逐字节核验 hash 绑定的 generated_finals_results.tex, 然后才调用 XeLaTeX/BibTeX。第2表、第3表和本节所有数值宏都来自该导入文件。

表 2: 固定 Mac 2 GiB/4 vCPU/no-swap 协议下的两轮聚合结果 (由 finals-results.json 派生)。

配置	缓存	峰值内存 (GiB)	Wall (s)	Decode (t/s)	专家浪费 (B)
baseline	cold	2.000	69.530	0.600	0
baseline	warm	2.000	53.535	1.232	0
patched-control	cold	2.000	66.300	0.689	0
patched-control	warm	2.000	60.300	0.821	0
patched-reclaim	cold	2.000	63.390	0.732	0
patched-reclaim	warm	2.000	58.080	0.844	0
patched-residency	cold	2.000	66.160	0.692	0
patched-residency	warm	2.000	63.190	0.712	0
patched-combined	cold	2.000	79.525	0.668	0
patched-combined	warm	2.000	63.265	0.760	0

表 3: 由两轮 cache-separated promotion gate 聚合的功能分类 (由 finals-results.json 派生)。

机制	cold	warm	总体
错误专家回收	kept_opt_in	kept_opt_in	kept_opt_in
专家驻留	kept_opt_in	kept_opt_in	kept_opt_in
组合策略	rejected	kept_opt_in	rejected

finals-results.json SHA-256: 843b54ada66bfea0d52ec55efcbf8f91b592d588cbffb0204efc40400196c655

5.4 赛前末次 A24 运行复核

2026 年 8 月 17 日又执行了一次单独的终态复核: 仍使用 48,410,988,384 B 的 Qwen3-Next-80B-A3B-Instruct Q4_K_M、2 GiB cgroup、4 vCPU、no-swap、pp64/tg16 和 cold cache, 并开启动态阶段建议, 同时设置 SLIM_ARC_NO_EXPERT_PREFETCH=1。该运行成功退出, 没有 OOM; Prefill 为 3.908455 t/s, Decode 为 0.709095 t/s, wall time 为 58.06 s, memory.peak 为 2 GiB, major faults 为 402,118。运行时记录了 816 个专家样本, 而 expert advice requests、issued bytes 和 waste bytes 均为 0, 说明 A24 精确关闭专家投机预取的开关在最终镜像中生效。

该复核只有一次运行, 而且镜像与 8 月 12 日正式矩阵不同, 因此只用作“二进制、资源边界和开关仍然可运行”的终态证据, 不参与表2的配置排序, 也不与旧行相除形成新加速比。结构化控制器结果、llama-bench JSON、GNU time、cgroup 和运行时指标保存在 docs/macros_test_notes/2026-08-17/ecospec-a24-final-2g-4c-pp64-tg16-cold/。

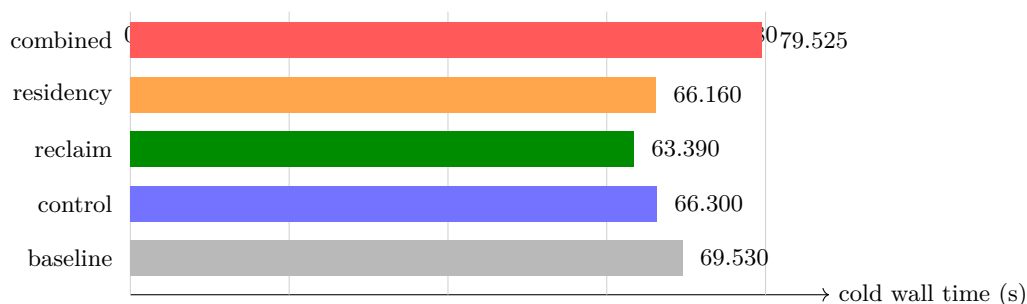


图 2: 固定 2 GiB 协议下的 cold wall time。数值由 hash 绑定的正式 JSON 宏生成；该图用于展示配置排序和组合回退，不表示跨设备加速比。

5.5 跨设备边界实验

Mac 正式矩阵回答“在严格可复现的 2 GiB 容器协议下是否应推广新机制”；RK3588 与树莓派实验回答“页建议和提前读在不同存储链路上为什么会改变方向”。三者的 CPU、文件系统、缓存边界和工作负载不同，因此表 4 只在每台设备内部做匹配比较，不计算跨设备加速比。

表 4: 决赛跨设备实验及其可推广边界

环境	匹配实验	观测	决策
Mac/Colima, 2 GiB, no-swap	两轮五配置 pp64/tg16	cold/warm, 20/20 成功; 所有配置峰值均为 2 GiB; 组合策略 cold wall 回退 19.947%	回收/驻留保留 opt-in; 组合拒绝
RK3588, 8 GB, NVMe	80B, p32/n16/t4; 静态 RAN-DOM、禁用、动态阶段建议	静态全开为 0.44/0.26 t/s; 禁用为 2.74/1.41 t/s; 动态策略达到 2.84/1.40 t/s	Prefill 使用 SEQUENTIAL; Decode 在该设备保留预读, 拒绝静态 RANDOM
树莓派 5, 4 GiB, NTFS-3G/FUSE, no-swap	A24-A50: resident cache、统一预算、预取开关、替换策略、文件系统与 top-k	A28 相对 A24 decode +14.16%; A29 投机 weight I/O -98.44%; A45 文件预读 -0.25%	常驻 shared/hot 路径并关闭投机预取; top-8 保留质量门

RK3588 的反例揭示了机制边界：当 45.09 GiB 权重在 8 GB 内存中持续轮转时，静态 MADV_RANDOM 把可合并的读请求分裂为大量同步小缺页；顺序预读通过较大 I/O 摊薄内核 fault、DMA 与 SMMU 以及弱核中断路径上的固定开销。阶段感知策略恢复了 Prefill，并使 Decode 追平禁用路径。这里的结论是“策略匹配硬件”，而不是“SEQUENTIAL 在所有设备上更快”。

树莓派实验使用同一 48,410,988,384 B 模型和相邻开/关运行。POSIX_FADV_WILLNEED 只把请求提前交给 FUSE/内核，没有减少实际读取量，也没有改善 USB Bulk-Only 队列深度，因而结果中性略负。A47 对旧源码的复现与当前 control 只差 0.61%，却无法复现历史所谓 cold 绝对值，说明仅清 Linux page cache 不能证明 FUSE、USB bridge 和 SSD 内部缓存同时为 cold；相关历史跨批次倍数不用于结论。

5.6 Ascend 910B4 最小兼容性验证

项目通过 HiDevlab 跳板 SSH 在独立目录完成一次最小硬件与软件栈探测。表 5 中的初始快照确认 910B4、驱动和 HBM 枚举正常，但该裸环境只有驱动，没有 CANN toolkit、libascendcl.so、系统 PyTorch 或 torch_npu。只读调用既有 Python 环境时又因缺少 libhcc1.so 停止，未执行算子。随后检测到另一个比赛的 NPU 进程开始占用 313 MiB HBM，本项目立即停止，不终止、不附着也不与其竞争资源。

表 5: Ascend 910B4 最小兼容性 smoke (非性能实验)

项目	观测
硬件	aarch64, Ascend 910B4, 32 GiB HBM, Health OK
驱动	npu-smi 25.2.0, ascendhal 7.35.23
初始空闲快照	AICore 0%, HBM 2889/32768 MiB, 89.9 W, 46 °C, 无 NPU 进程
用户态栈	Python 3.11.6; CANN/ACL/PyTorch/torch_npu 不完整
结论	仅证明 SSH、驱动和硬件枚举可用; 没有 LLM TPS、算子吞吐或 SLIM-ARC 加速结论

这项验证同时划清可移植边界: SLIM-ARC 的 POSIX/llama.cpp CPU 路径可以在 aarch64 Linux 上构建, 但 CANN 路径必须在匹配 toolkit、算子库和量化支持的独立镜像中重新构建与测量。当前 80B Q4_K_M 模型不能因“设备可枚举”就宣称已在 910B 上运行。

在用户新建的 HiDevLab 独立环境中, 项目进一步完成 OLMoE-1B-7B Q2_K 的系统开/关 A/B。该模型为真实 MoE, 但执行路径仍是 aarch64 llama.cpp CPU-only, 不使用 CANN 或 NPU。为使 2,562,763,232,B 小模型进入运行时, 本次隔离构建仅把 admission guard 从 6,GiB 降到 2,GiB; baseline 不改, 仓库默认阈值也不据此变更。模型 SHA-256、完整七组结果和 runtime 计数器保存于 docs/ascend_test_notes/2026-08-17/olmoe-q2-runtime-ab.json。

表 6: HiDevLab aarch64 CPU 路径上的 OLMoE 系统 A/B (非 NPU 性能)

配置	Prefill t/s	Decode t/s	Wall s	Max RSS KiB
llama.cpp baseline	100.724597	51.340323	3.254600	2,575,956
SLIM-ARC default	101.175889	45.254024	3.415535	2,577,888
SLIM-ARC A24	100.602680	48.184150	3.373611	2,577,380

两条 patched 结果都各自输出一条 schema 3 runtime 指标, 证明比较的不是误链接 baseline。A24 相对 default 的 Decode 提升 6.47%, wall 降低 1.23%; 但相对原生 baseline 仍为 Decode -6.15%、wall +3.66%。运行时记录了 785 个 expert samples 和 19,116 次 weight advice request, 但 expert_issued_bytes=0。因此, 本实验的正确结论是: SLIM-ARC 已在华为 aarch64 环境真实运行, A24 能缓解小模型上的调度开销; 内存充足、热缓存的 2.56,GB MoE 不是系统优势区间, 也没有专家预取或 NPU 加速结论。

5.7 树莓派慢存储专项优化

树莓派上的核心转向是从“尽量提前读”改为“只保留确定会复用的页, 并停止无收益的投机 I/O”。表7按实施顺序汇总有效机制和被否决方向。A24 是同一 campaign 的 demand-paging 参考; A28 将每层总会执行的 shared expert 和跨 token 复用的 hot experts 保留在内存, 避免每 token 重新穿过 FUSE/USB。A29 再用 16 MiB 统一预算限制 speculative weight advice; A32 证明将其完全关闭也不损失 wall time, 因此形成慢盘零投机默认候选。A34 的 512 MiB LRU 用有限驻留预算保留跨 token 页; 更复杂的 LFRU 虽改善部分 cache counters, 却增加非驻留扫描和 wall time, 因而被拒绝。

表 7: 树莓派 5 慢存储策略演进 (均为设备内匹配比较)

实验	机制	结果	决策
A26	512 MiB resident hot-expert cache	相对 A24: decode +9.19%, wall -4.48%; 两次运行 相差 1 s	保留
A28	A26 + 94.8 MB shared-expert mlock	相对 A24: prefill +1.45%, decode +14.16%, wall -6.58%	设备候选
A29	16 MiB unified speculative-I/O budget	相对 A28: 吞吐约 $\pm 0.2\%$; issued 与 in-flight bytes 均 -98.44%	推广到慢盘
A32/A33	关闭 weight/expert speculative advice	505 s 两次复现; 相对同环境 control wall 持平、decode +0.37%, 投机 I/O 清零	慢盘默认
A34/A35	512/384 MiB 跨 token LRU	512 MiB 相对旧 cache: evictions -36.31%, hits +6.53%, decode +0.93%; 384 MiB 反向	选 512 MiB
A37	LFRU 替换 LRU	prefill -1.60%, decode -1.99%, wall +1.50%, non- resident scan +56.33%	拒绝
A38-A40	hot admission threshold 由 1 提到 2	三个实现的 prefill 均约 -54%; recurrent microbatch 丢失首次命中后的复用	拒绝
A41/A42	NTFS-3G/FUSE 改为只读 NTFS3	1 MiB direct-read bandwidth +57.28%, 真实 mmap prefill 却 -65.70%	保留 FUSE
A45/A46	exact expert fadvise	prefill -0.251714%; 只提前请求, 未减少读取量	默认关闭
A49/A50	routed top-8 对 top-10, pp4/tg1	单次 prefill +17.56%, decode +10.53%, wall -3.30%	质量门前 opt-in

Top-8 的数据只代表性能上限, 不是默认配置。A51 与 A52 使用同一算术问题时, top-8 与原始 top-10 都回答 50%, 而正确答案为 46.67%; 该问题无法区分裁剪造成的质量变化。因此报告保留 A49/A50 的单次方向性数据, 同时要求后续 ARC-Challenge 多选准确率和重复 A/B 同时通过。这个边界体现了端侧优化的基本约束: 减少专家计算与 I/O 可以直接换取性能, 但必须把质量损失作为一等预算。

6 端侧 Agent Harness: 面向受限推理的工作负载闭环

本赛题关注模型运行时, 但实际端侧 Agent 并不是一次性的问答请求。模型需要生成工具调用、等待工具执行、回填观测, 再继续生成; 如果每一步都重新加载权重, 或者把完整终端输出反复塞回上下文, 运行时节省下来的内存和 I/O 很快会被编排层重新消耗。为此, SLIM-ARC 增加了一个最小 Agent Harness, 把受限内存推理能力接到可复现的工具闭环中。该模块不改变模型计算图, 其作用是约束上层请求, 使运行时优化在多轮负载中仍然有效。

6.1 实现路径

Harness 复用常驻的 OpenAI-compatible HTTP endpoint, 执行 `model` $\rightarrow$ `shell` $\rightarrow$ `observation` $\rightarrow$ `model` 循环。模型只允许返回两种 JSON: `{"tool": "shell", "args": [...]}` 或 `{"final": "..."}` 。工具参数以 `argv` 数组传给子进程, 不经过 `shell` 解释器; 输出被裁剪后才作为 `observation` 回填。稳定 `system prefix` 始终保留, 其余消息从最近轮次向前装入字符预算, 从源头限制后续 Prefill 的增长。

常驻 endpoint 是正式执行路径。仓库同时保留 `llama-cli` 兼容入口, 便于没有服务端时做功能排查, 但该入口每个 Agent step 都可能重新加载模型, 因此不用于性能结论。当前实现的边界和指标见表8。

表 8: Edge Agent Harness 的端侧约束策略

端侧问题	Harness 策略	默认边界与可观测量
重复加载权重	复用常驻 HTTP 模型服务	记录每轮请求耗时；CLI fallback 不作为性能路径
工具轮次扩大上下文	稳定 prefix + 最近消息窗口；工具观测居中截断	context 8192 字符，单次 observation 2048 字符
Decode 较慢、回答过长	根据已观测的输出速度缩减下一轮 max_tokens	低于 2.0 t/s 时乘 0.65；192 tokens 最低降到 48
工具阻塞或输出爆炸	命令白名单、固定工作目录、超时和输出上限	默认 8s、4 KiB；记录返回码、耗时和截断标志
多轮失控	Agent step 与总墙钟时间双重上限	默认 4 steps、300s；超限显式失败

6.2 短上下文与慢输出自适应

上下文管理采用固定前缀和有界最近窗口。Harness 不做摘要模型调用，也不引入新的常驻内存结构；当消息超过 8192 字符时，优先保留 system prefix 和最近的完整消息，工具输出单独限制在 2048 字符。这样可以给每轮 Prefill 一个确定上界，同时保留最近一次工具结果。

输出预算由上一轮观测驱动。初始上限为 192 tokens；若估算速度低于 2.0 t/s，下一轮按 0.65 倍收缩，并设置 48 tokens 的下限。这个策略不改变模型采样结果的概率分布，只限制允许生成的最大长度；它适合磁盘缺页或弱 CPU 导致的慢 Decode 场景。若任务确实需要长输出，用户仍可显式提高预算。

6.3 受限 Shell 与可观测性

默认命令只包含 pwd、uname、df、npu-smi 等只读诊断操作。Harness 拒绝非白名单程序、绝对路径、.. 路径和越出工作目录的符号链接，不使用 shell=True，也不把认证头或环境变量写入指标。每次工具执行均有独立超时和 4 KiB 输出上限。

运行指标使用 JSONL 记录。模型事件包含上下文字符数、max_tokens、请求耗时、TTFT、估算输出 token 数与 TPS；工具事件包含 argv、返回码、耗时、输出字符数和截断标志。两类记录可以把“模型慢”与“工具慢”分开，避免把编排等待误归因于推理运行时。

6.4 功能闭环验证

图3是一次确定性本地 smoke 的实际终端记录。测试 endpoint 先返回 uname -m 工具请求，Harness 在受限 shell 中执行并回填观测，第二轮再返回最终答案。首轮估算速度为 1.999 t/s，触发预算从 192 降到 124，下降 35.4%；工具返回码为 0，输出未截断。若后续轮次持续低于阈值，预算最多可由 192 降到 48，即上限收缩 75%。这里验证的是策略闭环和边界是否生效，不是端侧模型的 TPS 加速结果。

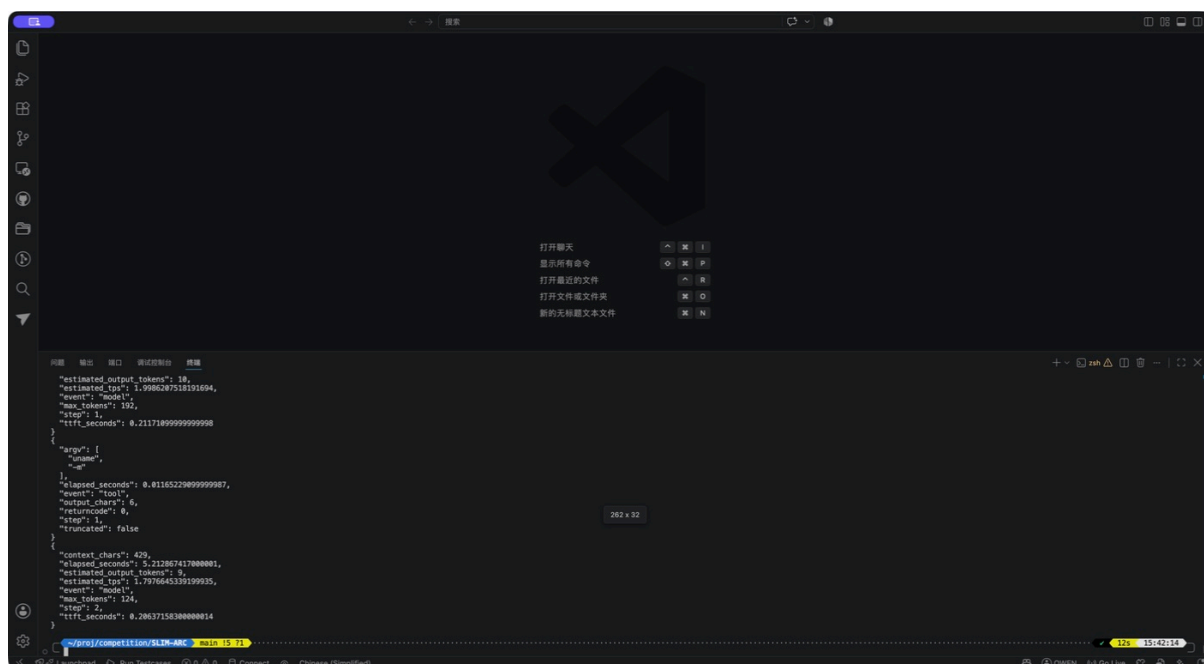


图 3: Agent Harness 确定性 smoke 的实际 JSONL 记录。测试使用本地 OpenAI-compatible SSE endpoint, 依次完成模型决策、受限 `uname -m` 工具调用和最终回答。该图验证工具闭环、指标记录和自适应 token 上限, 不代表真实 80B 模型性能。

仓库中的 10 个单元测试进一步覆盖上下文裁剪、慢输出预算调整、命令与路径拒绝、输出截断, 以及 fake model 驱动的完整闭环。尚未完成固定端侧模型上的 Harness 开/关 A/B, 因此本节不宣称 TTFT、TPS 或峰值内存收益。当前可确认的是: 运行时负责减少权重与专家访问成本, Harness 负责控制轮数、上下文和输出长度; 两者的指标口径已经分开, 后续可以在同一设备上做端到端归因。

7 结论、局限性与未来工作

SLIM-ARC 的决赛核心贡献不是某一个固定 madvise 参数, 也不是无条件的专家预取, 而是基于阶段、Router 反馈和内存压力的专家权重管理闭环。阶段策略决定是否保留内核预读; 专家预测是可撤回、受预算约束的建议; 错误预取只对经过边界验证的内部页回收; 驻留仅在压力、预测浪费和预算允许时准入; 模型所有权与租约定义后台工作和映射地址的共同生命周期。边界条件、重复认领、压力状态、指标解析和打包变体均有自动化验证; 正式实验进一步把每个成功运行绑定到源码、patched tree、模型、镜像与 cgroup 证据。

端侧 Agent Harness 进一步把运行时能力接到多轮工具负载: 常驻 endpoint 避免每步重载, 短上下文和慢输出预算限制编排开销, 受限 shell 提供可演示闭环。当前证据已经验证工具循环、预算收缩和指标记录, 但尚未完成固定端侧模型上的 Harness 开/关 A/B, 因此不把它写成新的加速结论。

其局限性同样明确。页建议是 best-effort, 内核、FUSE、USB bridge 和设备缓存会使所谓 cold 状态难以复现; 保守的页对齐会损失覆盖率; 压力阈值和预测信号可能不适用于所有模型或工作负载; 单一 2 GiB/no-swap、短生成 Mac 矩阵不能推广到其他设备或长上下文; 所有配置都触及同一 2 GiB 峰值, 无法证明峰值内存下降; combined cold 回退 19.947%, 说明组合并非天然协同。树莓派上的提前文件读没有收益, 而“常驻确定复用路径 + 限制或关闭投机 I/O”有效, 说明慢存储优化

的第一目标应是减少无效读取字节，而不是仅把同一批 I/O 提前。

HiDevLab 的 OLMoE A/B 又给出一个互补边界：在 aarch64 CPU-only、2.56 GB 模型、内存充足和热页缓存下，A24 虽比默认 runtime 的 Decode 高 6.47%，仍比原生 baseline 低 6.15%。该负面结果被保留，因为它说明系统的收益来自真实内存/存储压力，而不是 patched 二进制本身；同时也不能把 Ascend 开发环境误写成 CANN/NPU 加速。

比赛后的后续工作将沿三个可验证方向推进。第一，按专家连续打包并对 routed experts 采用更低比特，而保护 Router、Norm、Attention 与 shared expert，直接减少每 token 的磁盘读取量；第二，在真实 routing trace 上比较 per-layer LRU、LRU-K/ARC 与 workload-aware replacement，并报告 read bytes/token、cache hit、stall 和质量变化；第三，仅在固定质量集通过后评估自适应 expert top- k 或 PEU pruning。任何方向都必须分别报告 cold、warm 与 mixed cache，且不能把一个设备的最佳页建议静态迁移到另一设备。图1仍是设计闭环，性能解释只接受新的受控证据。

8 附录：可追溯性清单

8.1 初赛到决赛的里程碑

旧稿按六天计划罗列 Phase 0–5，且包含“留待决赛”等已经过期的状态。本版不再保留该开发计划，而以已经发生并可定位到提交或实验目录的里程碑替代，如表9所示。

表 9: 初赛到决赛的可验证里程碑

时间	阶段结果	决赛口径
6/21–6/26	完成初赛基线、mmap/MADV、量化、预取与初步消融	作为技术起点保留；跨缓存、跨设备或审计失效的倍数不沿用
8/5–8/10	在 RK3588 上跑通 80B，并定位静态 RANDOM 的严重负优化	实现阶段感知建议；动态策略在匹配工作负载下追平禁用路径
8/11–8/12	完成 Mac 2 GiB/no-swap 身份绑定矩阵	20 次成功运行由同一 JSON 导入；回收/驻留 opt-in，组合拒绝
8/13–8/17	转向真实边缘设备与慢存储链路	树莓派形成 shared/hot 常驻、16 MiB 预算、零投机预取与 512 MiB LRU 的设备策略；top-8 保留质量门
8/17	完成 Ascend 探测及 HiDevLab OLMoE A/B	旧环境仅做硬件枚举；新独立环境跑通 aarch64 CPU-only baseline/SLIM-ARC，对小模型负面边界如实报告，不声称 NPU 加速

8.2 决赛阶段的主要问题与处理

问题	证据与处理
固定页建议不能跨硬件复用	RK3588 上静态 RANDOM 将 p32/n16/t4 降到 0.44/0.26 t/s；改为阶段感知 SEQUENTIAL/设备相关 Decode 策略后达到 2.84/1.40 t/s。
预取状态与 Router 结果可能错配	使用非复用 generation token；成功、空结果、缺节点和计算失败都必须结算或取消，避免记录泄漏与错误归因。
模型卸载与后台线程共享地址	将运行时归属于模型，图执行持有租约；销毁时拒绝新租约、等待在途租约并 join worker，再释放映射。
Mac patched 变体曾错误链接 baseline 库	旧 patched 性能行全部作废；正式矩阵在每次运行中绑定镜像、源码树、variant linkage、模型和 cgroup 证据。
慢 FUSE/USB 存储提前读无收益	树莓派 A45/A46 显示精确 expert fadvise 为 -0.251714% ；默认关闭，后续优先低比特专家、静态裁剪或缓存替换策略。
慢盘投机 I/O 大但吞吐无收益	A29 的 16 MiB 预算将 issued/in-flight weight bytes 降低 98.44% 且吞吐近似不变；A32/A33 进一步证明完全关闭 weight/expert speculative advice 后 wall 同为 505 s。
热专家 cache 需要匹配 re-current 访问	512 MiB LRU 将 evictions 降低 36.31% 并保持吞吐；LFRU 增加 nonresident scan 56.33%，两次命中准入使 prefill 下降约 54%，均被拒绝。

8.3 提交材料与代码入口

材料入口如下；初赛报告目录保持独立，不被本次修订覆盖。

- 核心运行时: patches/llama-upstream/slim-arc-*;
- 幂等集成脚本: scripts/apply-slim-arc.py;
- Mac 控制器与证据工具: scripts/macros/;
- 端侧记录: docs/rk3588_test_notes/ 与 docs/pi5_4GB_test_notes/;
- Ascend 探测与 OLMoE A/B: docs/ascend_test_notes/2026-08-17/;
- 决赛报告: reports/Competition_Report_Finals/。

8.4 实验可追溯性

对象	决赛要求
源码与镜像	固定 commit、patched source hash、llama.cpp 360e134、镜像 ID 和 variant linkage。
模型	固定文件名、大小和 SHA-256；挂载为只读。
资源边界	2 GiB、4 vCPU、no-swap 的 cgroup 文件记录；无有限值的行不接受。
工作负载	pp64/tg16、prompt、seed、context 与 cold/warm 标签保持可比较。
结果行	outcome、wall time、吞吐、cgroup peak、fault/I/O 和每个 patched 行的唯一运行时指标。
负面结果	OOM、timeout、error 及未通过 promotion gate 的配置保留在 JSON 和总结中。
报告导入	只从机器生成的 finals-results.json 及其 raw manifests 写入最终表格或图；不得手工转录。

8.5 报告构建保护

当固定路径的正式 JSON 不存在、无效、不是恰好两轮完整 20 次成功运行，或生成的 TeX 与当前 JSON hash 不一致时，构建驱动将以明确错误退出，不留下可能被误交付的“final”PDF。该错误是发布门，而不是 TeX 环境错误。本次输入已到位并通过该门禁；维护者在结果文件变化后仍必须按 RESULTS_IMPORT.md 重新运行构建驱动，并对新 PDF 逐页检查。

参考文献

- [1] H. Du, S. Wu, A. Kharlamova, N. Guan, and C. J. Xue, “Flexinfer: Breaking memory constraint via flexible and efficient offloading for on-device LLM inference,” in *Proceedings of the 5th Workshop on Machine Learning and Systems*. ACM, 2025, pp. 56–65.
- [2] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, “Powerinfer-2: Fast large language model inference on a smartphone,” in *Proceedings of the European Conference on Computer Systems*, 2024, bibliographic entry checked against the locally archived manuscript.
- [3] X. Xia, J. Liu, X. Hou, P. Tang, M. Zhang, W. Wang, and C. Li, “Moe-prism: Disentangling monolithic experts for elastic MoE services via model-system co-designs,” arXiv preprint, 2025, locally archived manuscript; used only as design motivation.
- [4] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations*, 2017.
- [5] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, pp. 1–39, 2022.
- [6] J. Xie, R. Liu, H. Huang, Y. Ling, H. Dai, Y. Zheng, and W. Hu, “Less experts, faster decoding: Cost-aware speculative decoding for mixture-of-experts,” 2026.
- [7] H. Chen *et al.*, “KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models,” in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025.